Docker 部署语言 · C/C++ 与 Java

第 10 讲

目标

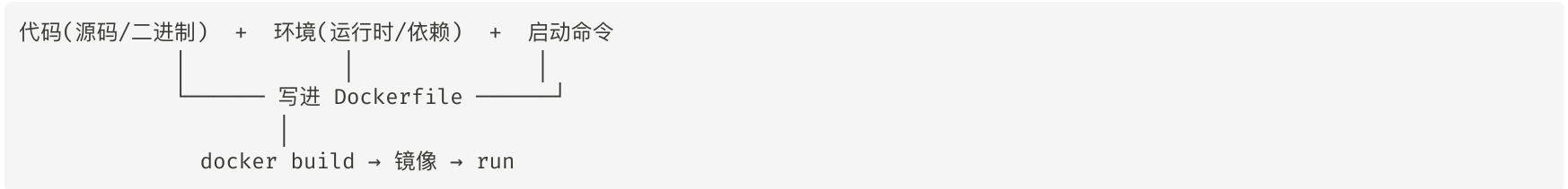
用 Dockerfile, 把程序和环境一起打包成镜像

- 部署心法: **代码 + 环境 → 镜像 → build → run**
- 编译型难点: C 用**多阶段构建**
- Java 部署: JDK/JRE · 打 jar · jre 运行
- 知识拓展: `ENV` 环境变量 + `JAVA_HOME`

一、部署的心法

> 程序不是裸奔的, 它是"程序+环境"

部署 = Dockerfile 打三个东西





> 难题在"环境", 各语言不一样

编译型多一个"先编译"步骤

- C/C++: `.c` → 编译器 → 可执行二进制
- Java: `.java` → javac → `.class` → jar → JVM 运行
- 二进制/字节码**依赖编译时的环境**

> 最稳做法: 在构建镜像里编译, 再拷产物到精简镜像 —— **多阶段构建**

二、多阶段构建

> 编译的大镜像, 运行瘦下来

一个 Dockerfile 多个 FROM, 只有最后的产物留下

```
FROM gcc:12 AS build          # 阶段一: 编译工具人
WORKDIR /src
COPY hello.c .
RUN gcc -o hello hello.c

FROM debian:bookworm-slim     # 阶段二: 精简运行
COPY --from=build /src/hello /usr/local/bin/hello
CMD ["hello"]
```



> COPY --from=阶段名 只把编译产物带进最终镜像

多阶段的好处: 最终镜像瘦

```
阶段一(build): gcc + 头文件 + 编译产物   →  ~1GB, 不要了
阶段二(runtime): 只放编译好的二进制      →  ~几十MB, 保留
```



> 环境与产物分清, 镜像小而稳

三、部署 C 程序

> 一波带走

C 部署三连: 编译 → 拷产物 → 运行

```
FROM gcc:12 AS build
WORKDIR /src
COPY hello.c .
RUN gcc -o hello hello.c

FROM debian:bookworm-slim
COPY --from=build /src/hello /usr/local/bin/hello
CMD ["hello"]
```



```
docker build -t hello-c .
docker run --rm hello-c    # C in Docker OK
```



四、部署 Java (重点)

> 先从"中间产物"认清 Java

Java 也要拆成 编译 + 运行

阶段	需要	干什么
构建	**JDK**(含 javac)	编译 .class → 打 jar
运行	**JRE**(只有JVM)	java -jar 跑起来

> 官方镜像: **eclipse-temurin**-17-jdk / -17-jre

形态A: 简单源码编译运行

```
FROM eclipse-temurin:17-jdk AS build
WORKDIR /app
COPY Hello.java .
RUN javac Hello.java

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/Hello.class .
CMD ["java", "Hello"]
```



形态B: Maven 打 jar 再运行 (工程化)

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /build
COPY pom.xml .
RUN mvn dependency:go-offline        # 预下载依赖，用缓存
COPY src ./src
RUN mvn package -DskipTests          # 出 target/app.jar

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /build/target/*.jar app.jar
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```



知识拓展: 部署 Java 前的 ENV 环境

```
FROM eclipse-temurin:17-jre
ENV JAVA_HOME=/opt/java/openjdk \
    APP_HOME=/app
ENV TZ=Asia/Shanghai
WORKDIR $APP_HOME
COPY app.jar .
CMD ["java", "-jar", "app.jar"]
```

- `ENV` 把环境**固化进镜像**, 容器里也带得进去
- 个性化东西用 `docker run -e` 覆盖

```
docker run -e DB_URL=jdbc:mysql://db:3306/app my-java-app
```

五、实战一

部署 C 程序

> 多阶段一次走通

hello.c 进镜像, 一行命令跑起来

```
mkdir -p c-demo && cd c-demo
# hello.c: printf("C in Docker OK")

docker build -t hello-c .
docker run --rm hello-c        # C in Docker OK
docker images                  # 很小, 无编译器
```

六、实战二

部署 Java jar (作业)

> Maven 打 jar → jre 运行

一个读环境变量的 Java 应用

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /build
COPY pom.xml . && RUN mvn dependency:go-offline
COPY src ./src && RUN mvn package -DskipTests

FROM eclipse-temurin:17-jre
ENV WHO=world
WORKDIR /app
COPY --from=build /build/target/app.jar app.jar
CMD ["java", "-jar", "app.jar"]
```



```
// Hello.java
String who = System.getenv().getOrDefault("WHO", "world");
System.out.println("Hello " + who + " from Java!");
```



-e 一改, 同一个 jar 另一个说法

```
docker build -t hello-java .
docker run --rm hello-java               # Hello world from Java!
docker run --rm -e WHO=Docker hello-java # Hello Docker from Java!
docker images | grep hello-java          # jre 镜像, 很小
docker history hello-java                # 看不到 build 阶段文件
```



一张表带走全部要点

方面	要点
心法	**代码 + 环境 → Dockerfile → build → run**
编译型	需要"先编译" → **多阶段构建**
多阶段	多个 FROM, `COPY --from=` 只带走产物
C	gcc 编译 → slim 运行, 镜像几十MB
Java	JDK 编译 / JRE 运行 · Maven 打 jar · eclipse-temurin
ENV	`ENV` 固化进镜像 · `-e` 运行时覆盖 · JAVA_HOME

编译器重、运行时轻 – 镜像瘦身的心法, 一次学会管一年

> 下一步: 部署 Python 与 R —— 解释型, 没有"显式编译", 用官方运行时镜像 + pip 依赖清单, 更轻更短

谢谢大家

Q & A